

CREDIT CARD FRAUD DETECTION ANALYSIS

1. Import Required Libraries

```
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

2. Load the Dataset

```
from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).

import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/creditcard.csv')
```

3. Display the First 5 Rows

```
df.head()

{"type": "dataframe", "variable_name": "df"}
```

4. Check Dataset Shape

```
df.shape

(284807, 31)
```

5. Check Missing Values

```
df.isnull().sum()
```

```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

6. Check Class Distribution

```
df["Class"].value_counts()

# 0 = Genuine Transaction
# 1 = Fraudulent Transaction
Class
0      284315
1       492
Name: count, dtype: int64
```

7. Separate Features and Target Variable

```
# Target Variable
y = df["Class"]

# Feature Variables
X = df.drop("Class", axis=1)
```

8. Normalize the Data

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

9. Balance the Dataset Using SMOTE

```
smote = SMOTE(random_state=42)

X_balanced, y_balanced = smote.fit_resample(X_scaled, y)
pd.Series(y_balanced).value_counts()
Class
0      284315
1      284315
Name: count, dtype: int64
```

10. Split the Dataset into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
X_balanced, y_balanced, test_size=0.2,
random_state=42 )
```

11. Train the Logistic Regression Model

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=1000)
```

12. Make Predictions

```
y_pred = model.predict(X_test)
```

13. Calculate Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9500729824314581
```

14. Display Classification Report

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support		
0	0.93	0.98	0.95	56750		
1	0.97	0.92	0.95	56976		
	accuracy			0.95	113726	
macro avg		0.95	0.95	0.95	113726	
weighted avg		0.95	0.95	0.95	113726	

THANK YOU